

Watchdog

Universitatea din Bucuresti
Departamentul de Informatica
Instrumente si tehnici de baza in informatica

The Martian: CONSTANTIN ANDREI-GABRIEL
Grupa: 143

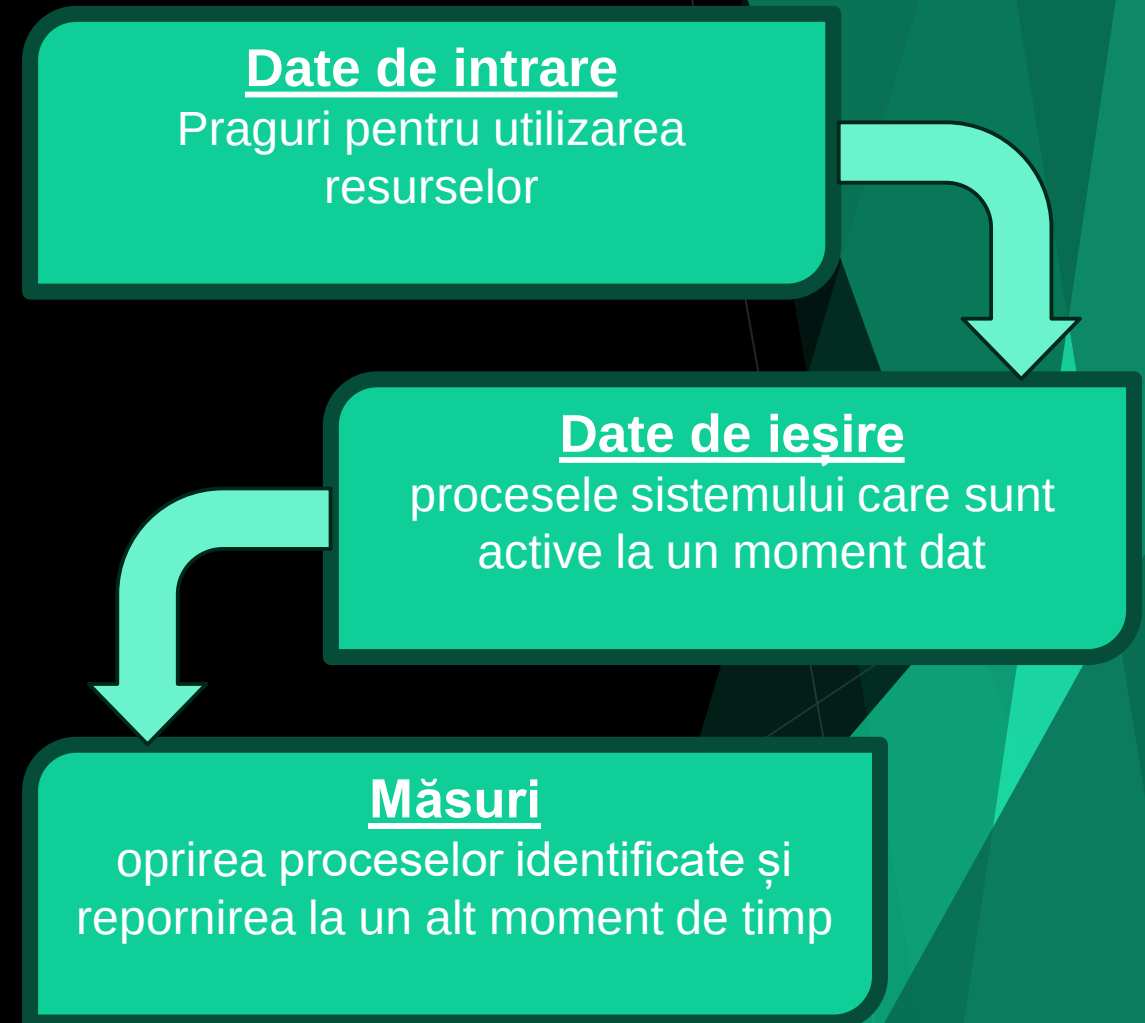
0x00) Outline

- ❑ Partea 1
 - ❑ 0x10) Decrierea problemei
 - ❑ 0x11) Importanța și scopul problemei
- ❑ Partea 2
 - ❑ 0x20) Specificația problemei
 - ❑ 0x21) Obiectivele proiectului
 - ❑ 0x22) Use-case-uri minimale
 - ❑ 0x23) Specificația prototipului
- ❑ Partea 3
 - ❑ 0x30) Design & UI
- ❑ Partea 4
 - ❑ 0x40)-0x4B) Implementare – shell script
- ❑ Partea 5
 - ❑ 0x50)-0x58) Implementare UI – Python

- ❑ Partea 6
 - ❑ 0x60) Experimente – arhitectura hardware/software
 - ❑ 0x61)-0x62) Experimente – proiect
 - ❑ 0x63)-0x64) Experimente – comparație
- ❑ Partea 7
 - ❑ 0x70)-0x72) Concluzii
- ❑ Partea 8
 - ❑ 0x80) Bibliografie

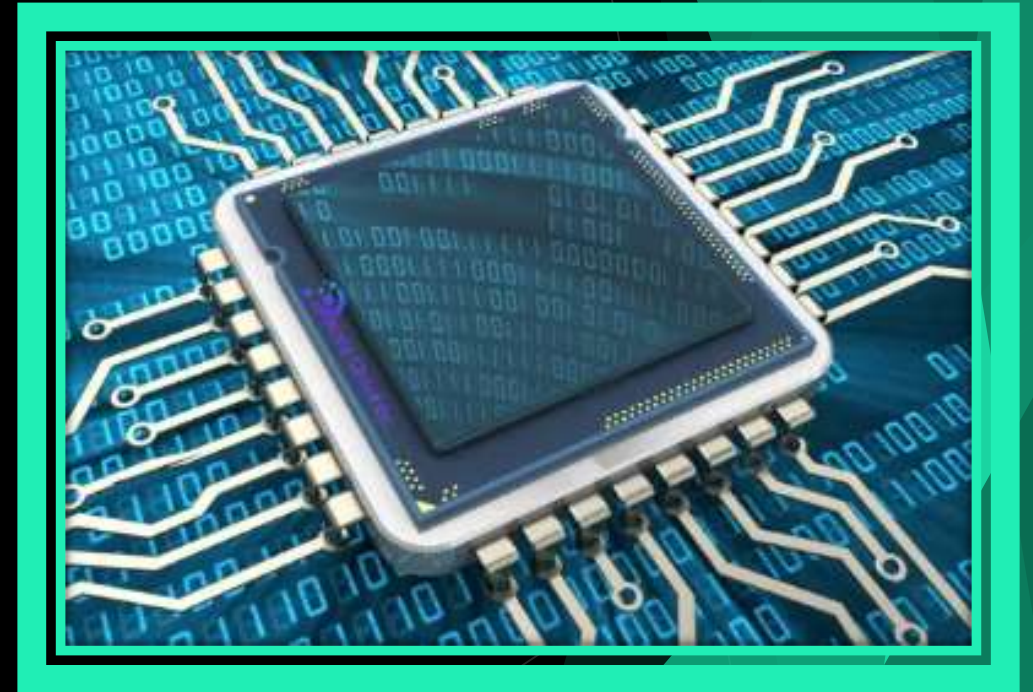
0x10) Descrierea problemei

- ❑ Ce proces utilizează intensiv resursele sistemului? Cum identificăm suspectii și cum remediem încetinirea sistemului nostru?
- ❑ Dorim să realizăm un script shell care monitorizează procesele active la un interval de timp prestabilit și verifică dacă există procese care au depășit pragurile impuse (asupra utilizării CPU/RAM)
- ❑ Mai mult decât atât, vom lua măsuri împotriva proceselor care au depășit pragurile stabilite
- ❑ Totodată, vom crea o interfață UI prin care putem vizualiza aceste date printr-un program extern (scris în Python)



0x11) Scopul și importanța problemei

- ❑ Prevenirea supraîncărcării componentelor hardware și, implicit, a sistemului
- ❑ Identificarea proceselor problematice și impunerea unor limite
- ❑ Acționarea asupra acestor procese astfel încât să menținem stabilitatea sistemului
- ❑ Vizualizarea informațiilor despre sistem și procesele acestuia
- ❑ Automatizarea acestor “task-uri”



0x20) Specificația problemei

- ❑ Problema ne cere de fapt, să citim fișierele din toate directoarele de forma **/proc/PID**, unde PID=identificatorul unic al procesului. (acest lucru se face incredibil de ușor folosind comanda **top**)
- ❑ Apoi, după extragerea informațiilor despre procesele sistemului, trebuie să verificăm anumite valori specifice componentelor hardware.
- ❑ În final, trimitem **semnale** către aceste procese care au trecut verificările folosind comanda **kill**. *

```
andrei@constantin-andrei-X541NA:~/Desktop$ ls /proc
1      1433  1780  2069  281   41   499  94      locks
10     1435  18    2074  2816  416   5     acpi     mdstat
1004   1436  1812  2079  2817  42    50    asound   meminfo
1005   1437  182   2097  2826  43    500   bootconf misc
1006   1450  1828  2099  2839  431   51    buddyinf modules
1009   1465  1834  21    2890  44    52    bus      mounts
1025   1466  1839  2109  2891  440   53    cgroups  mtd
1026   1473  1846  2125  29    444   54    cmdline  mtrr
1035   1491  1848  2149  2923  445   55    consoles net
1040   1497  1851  2163  2944  446   56    cpuinfo  pagetypeinfo
1041   15    1856  2175  297   447   57    crypto   partitions
1042   1501  1858  2181  3     448   58    devices  pressure
1044   1503  1859  22    30    449   59    diskstats schedstat
1045   1514  1860  2204  300   45    6     dma      scsi
1046   1517  1864  2231  31    46    60    driver   self
1048   155   1866  2243  32    47    61    dynamic_debug slabinfo
1053   156   1867  2264  33    470   62    execdomains softirqs
```

```
andrei@constantin-andrei-X541NA:~/Desktop$ kill -l
1) SIGHUP      2) SIGINT      3) SIGQUIT     4) SIGILL      5) SIGTRAP
6) SIGABRT     7) SIGBUS     8) SIGFPE     9) SIGKILL     10) SIGUSR1
11) SIGSEGV    12) SIGUSR2    13) SIGPIPE    14) SIGALRM     15) SIGTERM
16) SIGSTKFLT  17) SIGCHLD   18) SIGCONT    19) SIGSTOP     20) SIGTSTP
21) SIGTTIN    22) SIGTTOU    23) SIGURG     24) SIGXCPU     25) SIGXFSZ
26) SIGVTALRM  27) SIGPROF   28) SIGWINCH    29) SIGIO        30) SIGPWR
31) SIGSYS     34) SIGRTMIN   35) SIGRTMIN+1  36) SIGRTMIN+2  37) SIGRTMIN+3
38) SIGRTMIN+4 39) SIGRTMIN+5 40) SIGRTMIN+6 41) SIGRTMIN+7 42) SIGRTMIN+8
43) SIGRTMIN+9 44) SIGRTMIN+10 45) SIGRTMIN+11 46) SIGRTMIN+12 47) SIGRTMIN+13
48) SIGRTMIN+14 49) SIGRTMIN+15 50) SIGRTMAX-14 51) SIGRTMAX-13 52) SIGRTMAX-12
53) SIGRTMAX-11 54) SIGRTMAX-10 55) SIGRTMAX-9  56) SIGRTMAX-8  57) SIGRTMAX-7
58) SIGRTMAX-6 59) SIGRTMAX-5 60) SIGRTMAX-4  61) SIGRTMAX-3  62) SIGRTMAX-2
63) SIGRTMAX-1 64) SIGRTMAX
```

* Comanda kill are mai multe semnale care controlează un proces cu un anumit PID. Unele semnale sunt folosite destul de des, precum SIGTERM, care este folosit și când apăsăm CTRL+C

0x21) Obiectivele proiectului

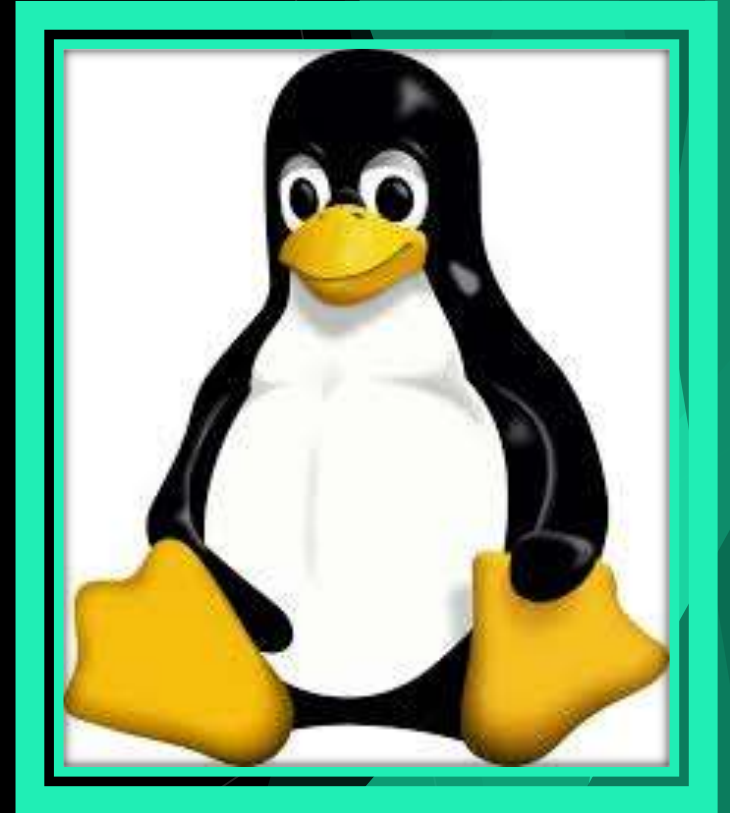
- ❑ Monitorizarea continuă a proceselor active în Linux
- ❑ Prevenția crash-urilor / freeze-urilor
- ❑ Performanță mai bună pe un sistem cu resurse limitate
 - ❑ Mai mult control asupra modului în care procesele afectează sistemul
 - ❑ Oprirea proceselor care folosesc prea multe resurse, astfel scăzând numărul de operații pe care procesorul le efectuează
 - ❑ Identificarea programe care îngreunează consistent sistemul
- ❑ Realizarea unei interfețe UI prin care utilizatorul poate observa cum sunt distribuite resursele sistemului către fiecare proces
- ❑ Personalizarea funcționalității scriptului prin command line (valori default și valori date ca input de utilizator)

0x22) Use-case-uri minimale

- ❑ Utilizatorul poate monitoriza un program creat/instalat pentru a testa performanța acestuia
- ❑ Utilizatorul poate observa procesele problematice care rulează pe sistem și consumă mult mai multe resurse comparativ cu alte procese
- ❑ Utilizatorul poate schimba pragurile ca să fie în concordanță cu sistemul acestuia
 - ❑ (Exemplu: Pe un laptop cu doar 4GB RAM utilizatorul poate seta pragul la doar 512MB)
- ❑ Vizualizarea utilizării procesorului și memoriei în timp prin intermediul graficelor din UI

0x24) Specificația prototipului

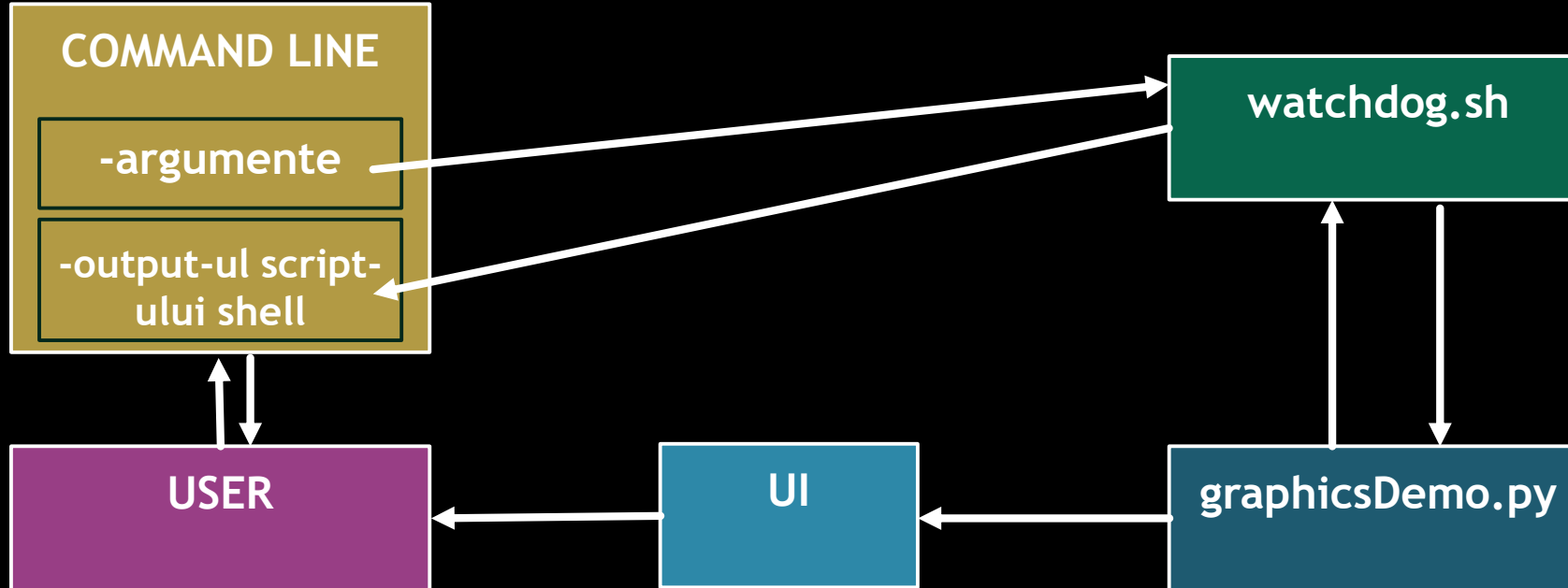
- ❑ Mediul de dezvoltare:
 - ❑ Linux Kernel
 - ❑ Distribuție: orice distribuție care suportă comanda top *
 - ❑ Comand Line
 - ❑ Terminal care poate rula script-uri folosind bash
 - ❑ Interpretor de Python instalat (python3)
 - ❑ Permisuni (**user / root**):
 - ❑ Rularea script-ului sub utilizatorul **root** **NU** va opri procesele care aparțin acestuia
 - ❑ Cerințe hardware recomandate:
 - ❑ Memorie RAM > 2GB (testat, fără probleme pe un laptop cu 4GB)
 - ❑ (programul utilizează ~5-10 MB când a fost testat)



* Am exclus comenzile standardizate de POSIX (vezi Bibliografie 0x23)

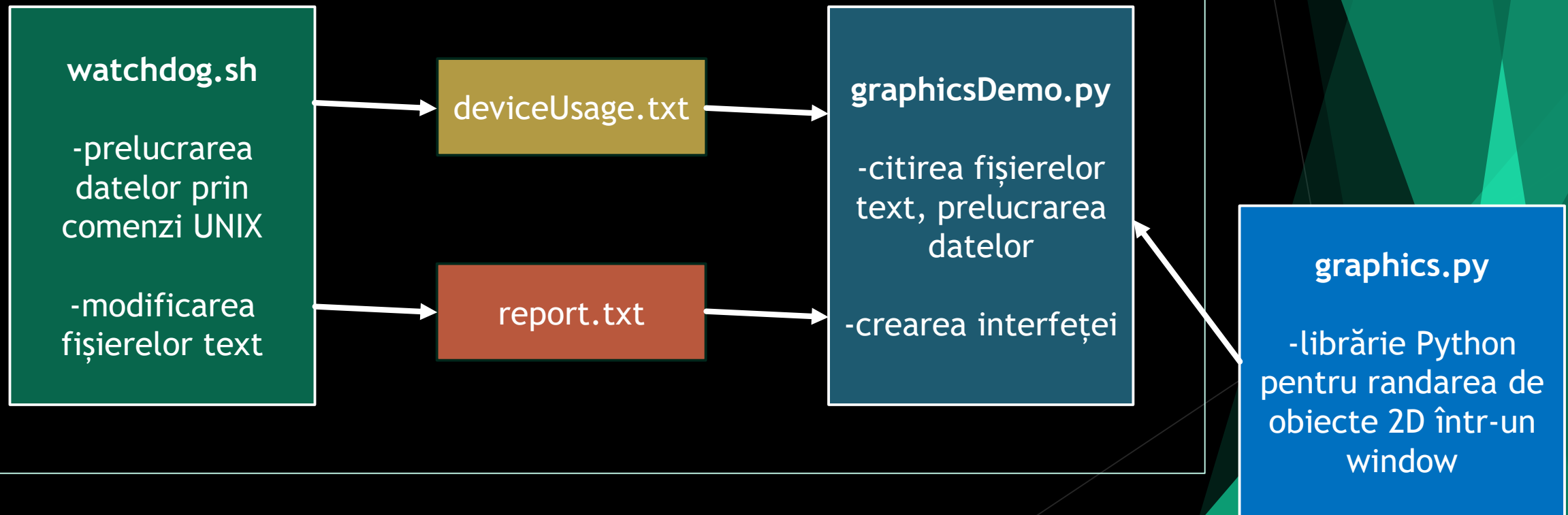
0x30) Design-ul prototipului

❑ COMUNICAREA ÎNTRE USER ȘI SCRIPT



0x31) Design prototipului

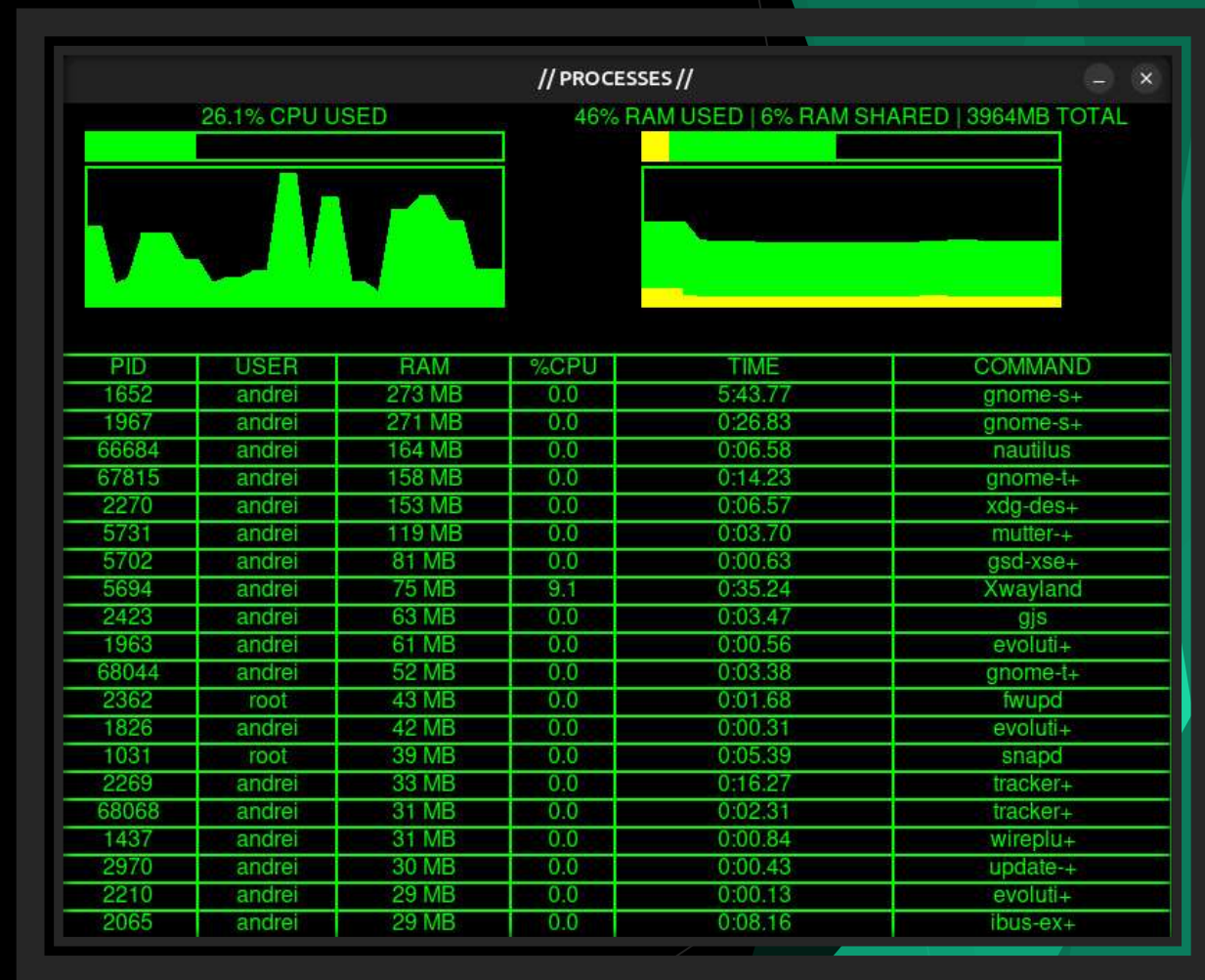
❑ COMUNICAREA ÎNTRE SCRIPT-URI



0x32) User Interface

- ❑ Interfața Python
- ❑ Utilizează un mic script open-source pentru grafică în Python*
- ❑ Grafic pentru monitorizarea procesorului și memoriei (residentă ne-swapată și cea shared)
- ❑ Datele sunt extrase în Python din fișierul “report.txt” care se află în directorul proiectului. Script-ul shell actualizează la un interval de timp fișierul

<https://mcsp.wartburg.edu/zelle/python/> *



Interfața în Python: Procesele sistemului sortate după consumul memoriei

0x40) Implementare - shell

- ❑ Abordarea prototipului:
 - ❑ Creăm un while loop care să execute o dată la o secundă comanda top
 - ❑ Vom folosi comanda top care va rula o singură dată (flag -n 1) în batch mode (flag -b) per iterație
 - ❑ Salvăm output-ul acestei comenzi într-un fisier text și iterăm apoi linie cu linie
 - ❑ (linie din fisier ⇔ informații despre process: PID, USER, %MEM, %CPU, RES, TIME, COMMAND etc.)
 - ❑ Verificăm pentru fiecare linie:
 - ❑ (memoria residentă) RES > 1024 (MiB) (adică 1 GiB)
 - ❑ (utilizarea procesorului) %CPU > 5.0
 - ❑ Dacă una din condiții este îndeplinită, omorâm procesul (comanda kill)

0x41) Implementare – shell

❑ SISTEMUL DE FIȘIERE /proc

- ❑ Putem optimiza comandă subliniată, astfel încât să nu mai apelăm la fiecare buclă comanda top care ne costă multe resurse

```
getDeviceUsage() {  
    CPU_USAGE=$( top -b -n 1 | head -n +5 | grep "Cpu(s)" | awk '{print 100 - $8}' )  
    FREE_RAM=$(free --mega | tail +2 | head -1)  
    echo $FREE_RAM | while read MEM TOTAL USED FREE SHARED CACHE AVAILABLE  
    do  
        P_USED=$(( $USED * 100 / $TOTAL ))  
        P_SHR=$(( $SHARED * 100 / $TOTAL ))  
        echo -e "\n $CPU_USAGE $P_USED $P_SHR $TOTAL" > deviceUsage.txt  
    done  
}  
  
USER=$(whoami)
```

Putem optimiza prin citirea fișierelor din /proc

0x42) Implementare – shell

- ❑ Optimizare (în versiunea 1.22 a proiectului)
- ❑ Vom folosi comenzile într-o buclă, deci trebuie să ne gândim la optimizarea timpului de rulare al acestei comenzi
- ❑ **Folosim comanda time pentru a compara cele 2 abordări**

```
andrei@UbuntuVM:~/Desktop/Watchdog---ITBI-main. (1)/Watchdog---ITBI-main$ time top  
-bn 1 | head -n +5 | grep "Cpu(s)" | awk '{print( 100 - $8 )}'  
5.8
```

```
real    0m0.406s  
user    0m0.027s  
sys     0m0.042s
```

```
andrei@UbuntuVM:~/Desktop/Watchdog---ITBI-main. (1)/Watchdog---ITBI-main$ time read  
-rs cpustxt user nice system idle iowait irq softirq x0 x1 x2 < /proc/stat
```

```
real    0m0.000s  
user    0m0.000s  
sys     0m0.000s
```

Această abordare este mult mai eficientă

```
andrei@UbuntuVM:~/Desktop/Watchdog---ITBI-main. (1)/Watchdog---ITBI-main$
```

0x43) Implementare – shell

- ❑ Condiția if pentru depășirea pragului de memorie (rezidentă / fizică)
 - ❑ Folosim variabila KB, definită la început ca (ramLimit << 10)
 - ❑ (transformare MB -> KB)
 - ❑ Folosim comanda kill cu semnalul default (adică -15 ⇔ SIGTERM)
 - ❑ De ce nu SIGSTOP? Pentru că SIGSTOP doar oprește procesul, iar resursele RAM sunt în continuare alocate procesului
 - ❑ De ce nu SIGKILL? Pentru că SIGKILL “omoară” abrupt procesul, și putem întâmpina erori

```
diff=$(( KB - ram ))
if [ $diff -lt 0 ]
then
    MBram=$(( ram >> 10 ))
    echo "Process $name (PID = $pid) RAM usage exceeded $ramLimitMB MB (USED: $MBram MB)"
    echo "Process $pid will be stopped (SIGTERM)"
    kill $pid
fi
```

0x44) Implementare – shell

- ❑ **Condiția if pentru depășirea pragului procesorului**
 - ❑ Pentru a reduce eventualele erori, am ales să tratez procentul ca un `int`
 - ❑ Ștergem punctul care delimitează partea fracționară de cea întreagă și împărțim la 10
 - ❑ De data asta folosim comanda `kill` cu semnalul -19 (SIGSTOP), pentru că vrem ca procesul să continue după ce operațiile făcute de CPU scad
 - ❑ Pentru continuarea procesului, punem un subshell în background care folosește comanda `kill -18 (SIGCONT)` după un interval de timp

```
cpu=$(echo $cpu | tr -d .)
cpu=$(( $cpu / 10 ))
diffCPU=$(( cpuLimit - cpu ))
if [ $diffCPU -lt 0 ]
then
    echo "Process $name (PID = $pid) CPU usage exceeded $cpuLimit% (Used: $cpu%)"
    echo "Process $pid will be stopped (SIGSTOP)"
    kill -19 $pid
    (sleep 10; kill -18 $pid; echo "Process $name (PID = $pid) running... (SIGCONT)")&
fi
```

0x50) Implementare UI - python

- ❑ Vrem să implementăm o tabelă care să afișeze un anumit număr de procese și informațiile lor organizate pe coloane
- ❑ Fiecare celulă are un text box care conține valoarea procesului din coloana i pentru fieldul de pe coloana j
- ❑ Informațiile le extragem din fișierele report.txt și deviceUsage.txt
 - ❑ Procesele din report.txt sunt sortate după RAM/CPU usage
- ❑ Ulterior, vom adăuga un text box pentru utilizarea RAM și CPU totală, dar și memoria shared
- ❑ În final, vrem să realizăm un grafic care monitorizează utilizarea RAM și CPU totală
 - ❑ Acesta va fi reprezentat de mai multe dreptunghiuri (sau trapezuri) aranjate una după alta cu înălțimea direct proporțională cu procentul resursei folosite

0x60) Experimente – arhitectura hardware/software

- ❑ Proiectul a fost atât dezvoltat cât și testat pe un laptop Asus cu următoarele specificații hardware:
 - ❑ Procesor Intel Celeron N3350
 - ❑ 2 core-uri
 - ❑ 2 thread-uri
 - ❑ Cache 2MB
 - ❑ Memorie RAM 4GiB
 - ❑ Stocare: Disk HDD
 - ❑ 500 GB
- ❑ Specificații software:
 - ❑ Kernel: Linux
 - ❑ Distribuție: Ubuntu 24.04.3 LTS

id:	memory
description:	System memory
physical id:	0
size:	4GiB

id:	cpu
product:	Intel(R) Celeron(R) CPU N3350 @ 1.10GHz
vendor:	Intel Corp.

CPU Specifications

Total Cores ?	2
Total Threads ?	2
Burst Frequency ?	2.40 GHz
Processor Base Frequency ?	1.10 GHz
Cache ?	2 MB L2 Cache

0x61) Experimente – arhitectura hardware/software

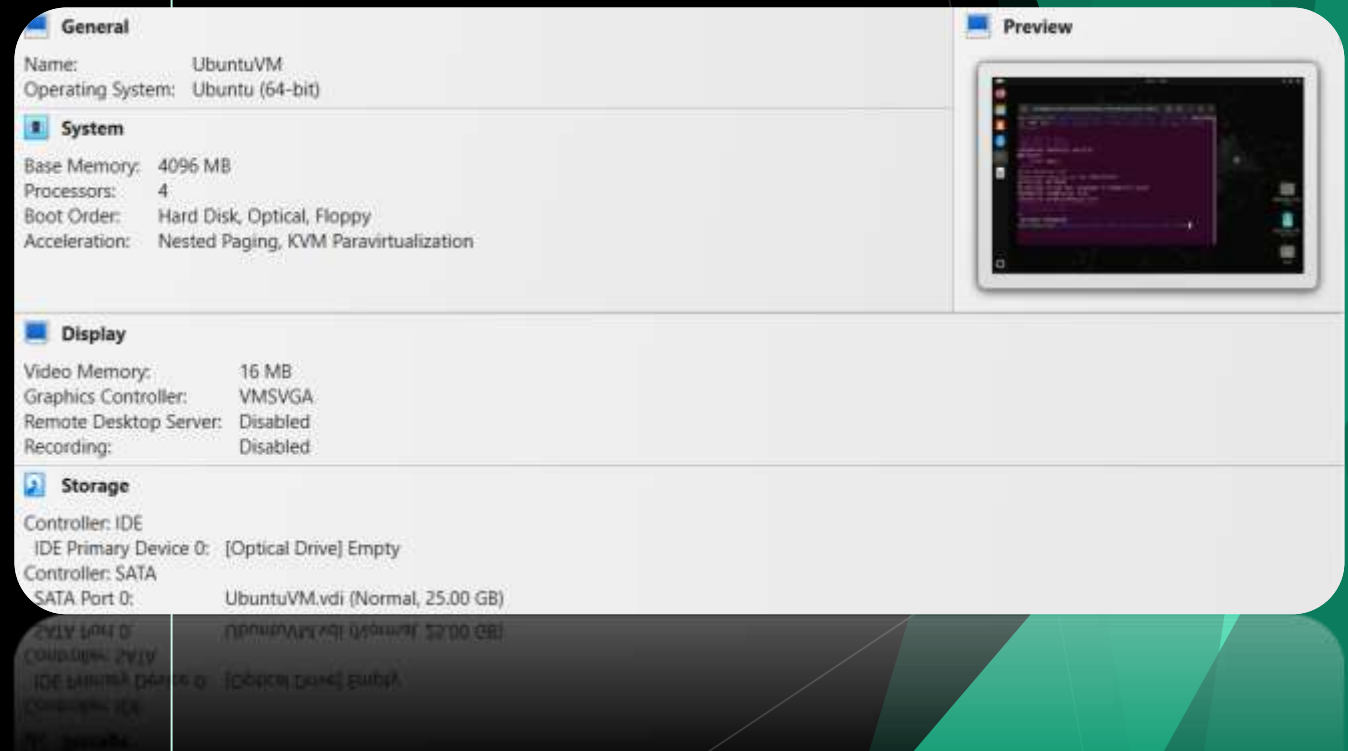
❑ De asemenea, pentru versiunile ulterioare ale proiectului, am testat script-ul in VirtualBox:

- ❑ 4 core-uri
- ❑ Memorie RAM 4GB

❑ Specificații software:

- ❑ Kernel: Linux
- ❑ Distribuție: Ubuntu 24.04.3 LTS

❑ Pentru experimentul realizat in VirtualBox, am ales sa rulez un benchmark online



<https://silver.urih.com/>

0x62) Experimente – proiect (V1.1)

- ❑ Am testat script-ul shell împotriva unui program Python dăunător

```
import random

v=[]

while True:
    v.insert(0,random.random()/random.random())
    pass
```

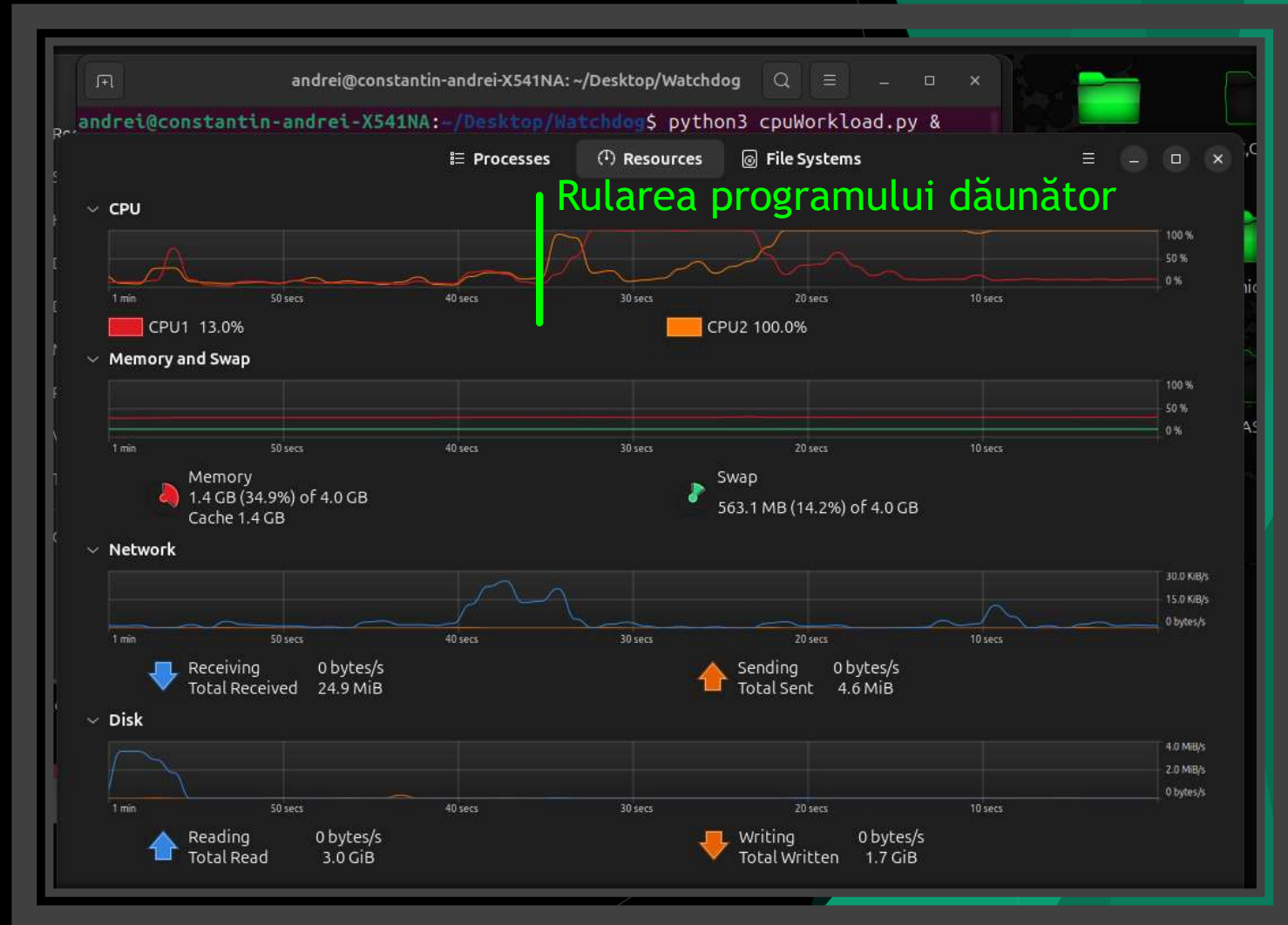
- ❑ De ce acest cod este dăunător?
 - ❑ Bucla infinită forțează realizarea repetată a operațiilor
 - ❑ Generarea numerelor aleatoare fracționare de foarte multe ori poate fi costisitoare
 - ❑ Împărțirea float-urilor este foarte dificilă pentru procesor
 - ❑ Adăugarea rezultatului la start-ul listei forțează mutarea cu un pas fiecare element următor
 - ❑ Adăugarea repetată a valorilor în listă va umple memoria RAM alocată programului la un moment dat

0x63) Experimente – proiect (V1.1)

- ❑ Vom rula acest program Python o dată fără script-ul pentru Watchdog și o dată cu acesta.

FĂRĂ WATCHDOG

- ❑ Imediat după rulare, utilizarea procesorului a unui core a ajuns la maxim
- ❑ Memoria RAM nu s-a modificat în timpul rulării
- ❑ Utilizarea medie totală a procesorului este în jur de 50%

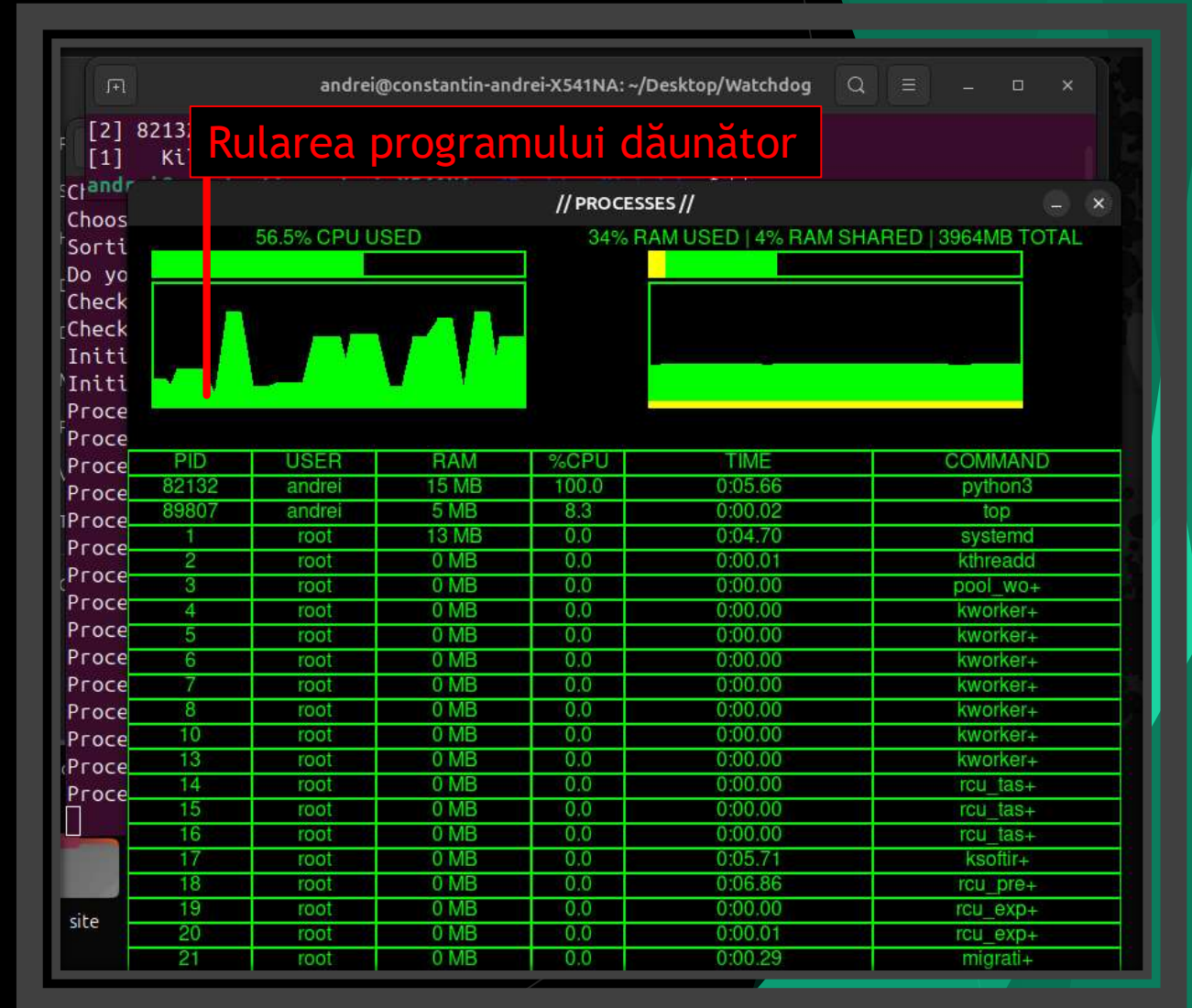


0x64) Experimente – proiect (V1.1)

- ❑ Vom rula acest program Python o dată fără script-ul pentru Watchdog și o dată cu acesta.

CU WATCHDOG

- ❑ Imediat după rulare, utilizarea totală a procesorului a crescut, deoarece un core a ajuns la maxim
- ❑ În intervalul de timp următor, programul este oprit, dar ulterior continuat
- ❑ De aceea avem aceste “spike-uri” în grafic
- ❑ Din nou, memoria RAM nu se modifică prea mult

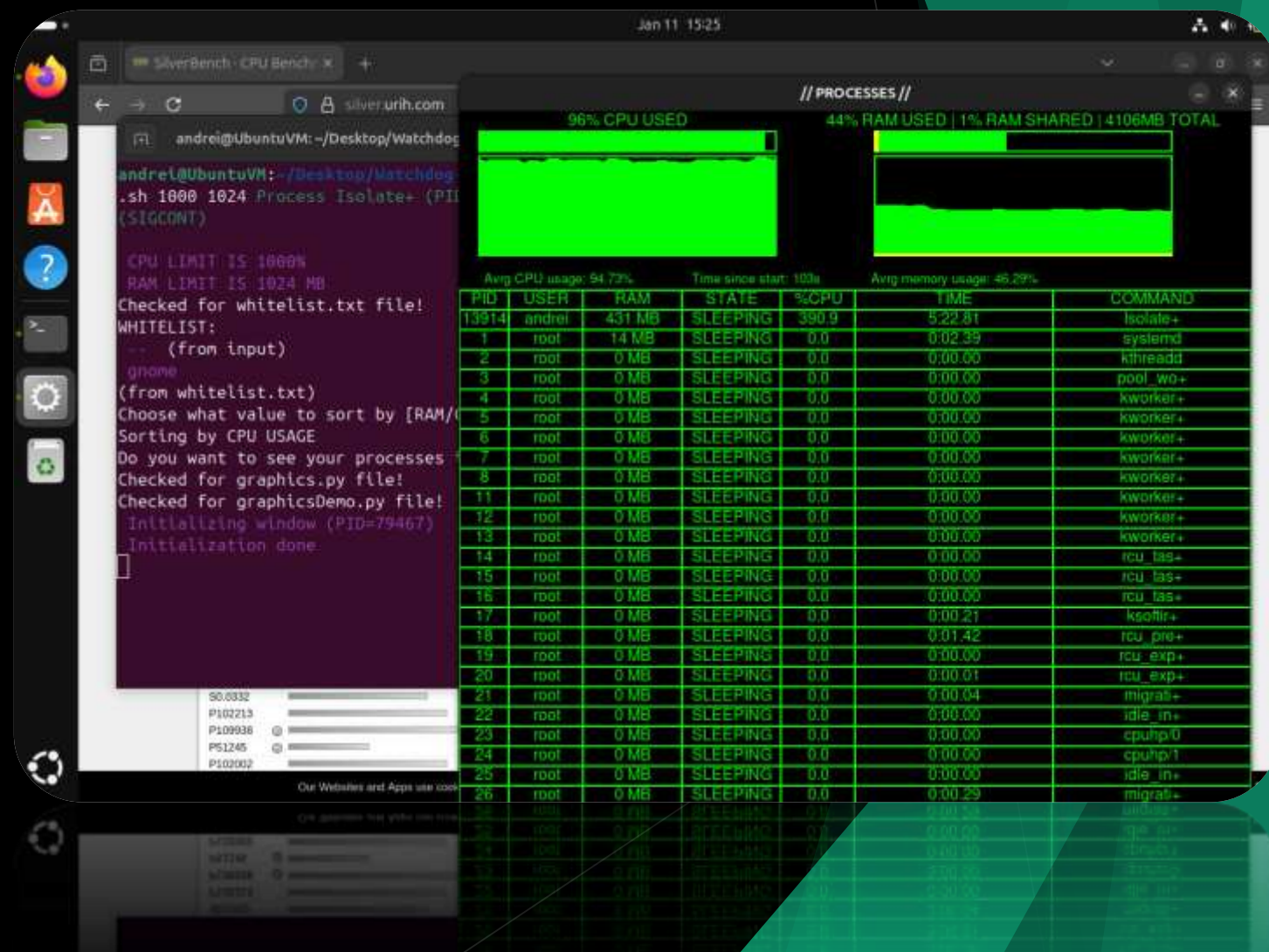


0x65) Experimente – (V1.22-UbuntuVM)

FĂRĂ WATCHDOG

(setam limita CPU la 1000%)

- ❑ Imediat după rulare, utilizarea procesorului ajunge la maxim
- ❑ Utilizarea medie a procesorului este în de 99%, si experimentam freeze-uri, mult mai rau decat in experimentul anterior
- ❑ Memoria RAM nu s-a modificat în timpul rulării

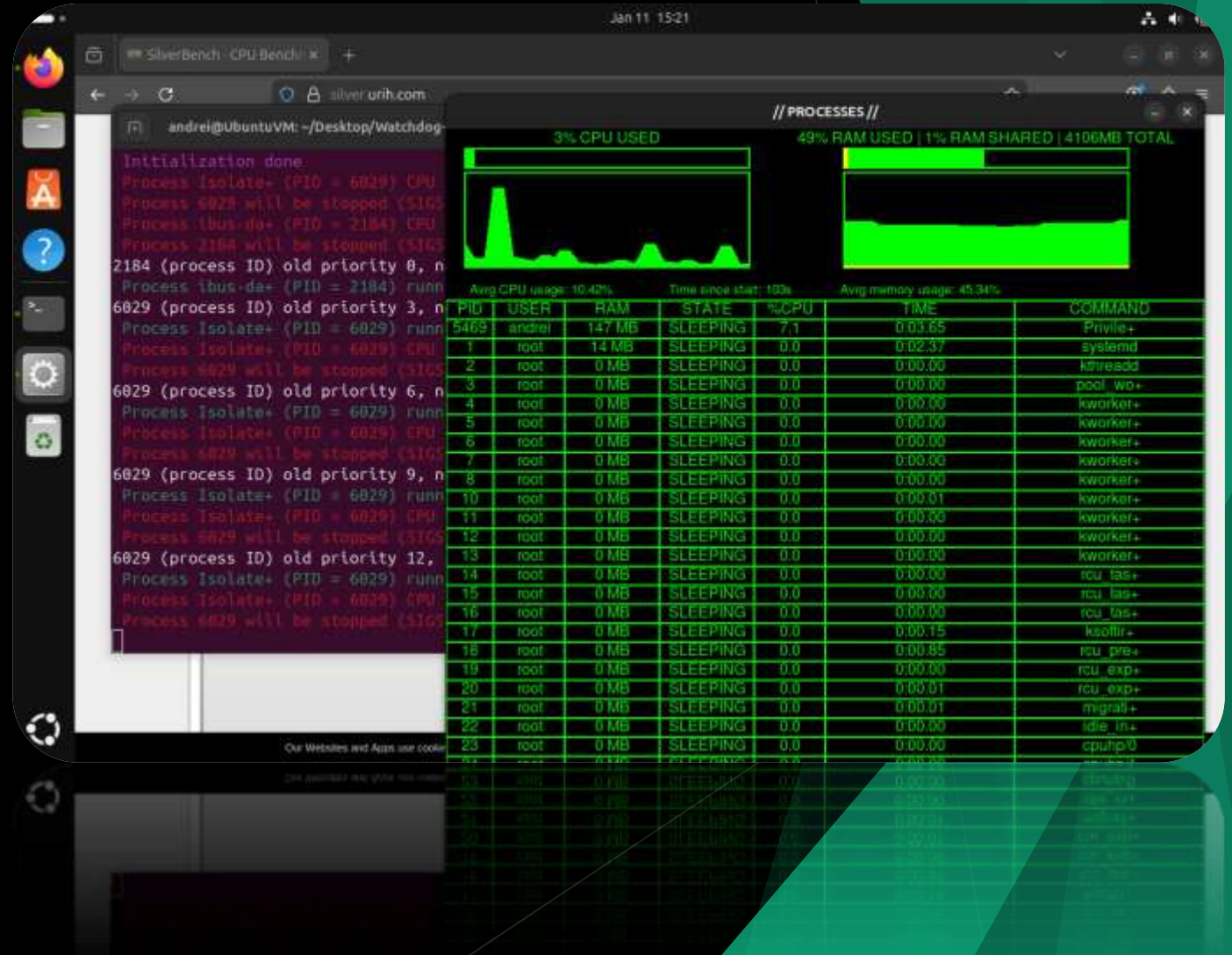


0x66) Experimente – (V1.22 - UbuntuVM)

- ❑ Vom realiza un benchmark online pentru procesor

CU WATCHDOG

- ❑ În intervalul de timp următor, programul este oprit, dar ulterior continuat, dar de data aceasta cu "pedepse" mai mari si delay-uri semnificativ mai mari
- ❑ De aceea avem aceste "spike-uri" în grafic, care sunt ceva mai rare
- ❑ Comparativ cu script-ul din experiment-ul trecut, acesta este mult mai eficient



0x67) Experimente – Comparație

FĂRĂ WATCHDOG

PRO

- ❑ Inițial nu vom experimenta “freeze-uri” și s-ar putea ca sistemul de operare să se ocupe automat de închiderea procesului dăunător (via crash)

CONS

- ❑ Nu știm programul defectuos, așteptăm să se rezolve problema de la sine
- ❑ Nu avem control asupra proceselor de pe sistem
- ❑ Trebuie să apelăm la aplicații externe ca să aflăm ce s-a întâmplat și să închidem manual procesul
 - ❑ (ex: System Monitor în Ubuntu sau Task Manager în Windows)

0x68) Experimente – Comparație

CU WATCHDOG

PRO

- ❑ Procesorul are spike-uri de workload, dar, în medie, utilizarea procesorului este ceva mai scăzută (V1.1) sau mult mai scăzută (V1.22)
- ❑ Putem observa programul dăunător și informații despre acesta
- ❑ Controlul pragurilor pentru CPU și RAM
- ❑ Pentru versiunea 1.22, incrementăm valorile "nice" ale proceselor, și, dacă ajung la maximul de 19, trimitem semnal SIGTERM către proces

CONS

- ❑ Script-ul shell nu va închide definitiv programul decât dacă trece peste limita impusă asupra memoriei (în experiment, programul utilizează 15MB)
- ❑ Comanda top a avut un workload semnificativ asupra procesorului (în experiment 5-10%)

0x70) Concluzii

- ❑ Proiectul oferă câteva funcționalități importante pentru oricine ce dorește să automatizeze management-ul proceselor de pe sistemul său
 - ❑ Limite impuse pentru RAM și CPU customizabile
 - ❑ Ordonarea proceselor după un criteriu (RAM/CPU)
 - ❑ Interfață grafică pentru vizualizarea mai bună a datelor
 - ❑ Scrierea unui whitelist cu keywords care să ignore procesele asociate
 - ❑ Output-ul script-ului în Command Line arată acțiunile luate
 - ❑ (Ar putea fi redirecționat către un log file)

0x71) Concluzii

- ❑ Ce am învățat prin acest proiect?
 - ❑ Obișnuirea cu script-urile shell și sintaxa lor
 - ❑ Concepte de shell scripting
 - ❑ (arithmetic expansion, pipelineing)
 - ❑ Cum vedem și controlăm procesele
 - ❑ Trimiterea în background
 - ❑ Folosirea comenzii top, kill, trap
 - ❑ Înțelegerea semnalelor folosite de comanda kill
 - ❑ Concepte de grafică și utilitățile bibliotecii graphics.py
 - ❑ Gestiunea datelor
 - ❑ Informații despre componentele hardware și funcția lor
 - ❑ Cum Linux organizează procesele în sistemul de fișiere /proc

0x72) Concluzii

- ❑ Ce aş îmbunătăţii la proiect?
 - ❑ Performanţa şi optimizarea codului
 - ❑ Pe viitor, o versiune care utilizează C++ pentru crearea unei interfeţe UI (ex: OpenGL) pentru grafică avansată
 - ❑ Comenzi integrate in script-ul shell pentru modificarea fişierului whitelist.txt
 - ❑ Continuarea proceselor în cazul în care utilizarea procesorului este sub o altă limită maximă de utilizare
 - ❑ Un “kill” count pentru fiecare proces
 - ❑ În funcţie de acesta să decidem dacă închidem definitiv procesul
 - ❑ (In versiunea 1.22 acest kill count este de fapt numarul de incrementari repetate al valorii "nice" necesare ca sa ajunga la maximul de 19, adica prioritate minima pentru procesor)

0x80) Bibliografie

❑ 0x20)

- ❑ <https://superuser.com/questions/243460/what-to-do-when-ctrl-c-cant-kill-a-process>
- ❑ <https://www.youtube.com/watch?v=TJzltwv7jJs>

❑ 0x23)

- ❑ <https://unix.stackexchange.com/questions/37064/which-are-the-standard-commands-available-in-every-linux-based-distribution>
- ❑ <https://pubs.opengroup.org/onlinepubs/9699919799/idx/utilities.html>

❑ 0x30)

- ❑ <https://mcsp.wartburg.edu/zelle/python/>
- ❑ https://www.youtube.com/playlist?list=PLmzkEJ1Zz_uZ5nvTOLaGHfinCzEVEVBlz

**Mulțumesc pentru
atenția acordată!**

<https://github.com/AndreiCd27/Watchdog-ITBI>